

Recap
 Q Learning
 Beh Policy/ Tgt Policy
 Deep Q Network
 Replay Buffer
 Stable Baseline (SB3)
 On Policy/ Off Policy
 SARSA

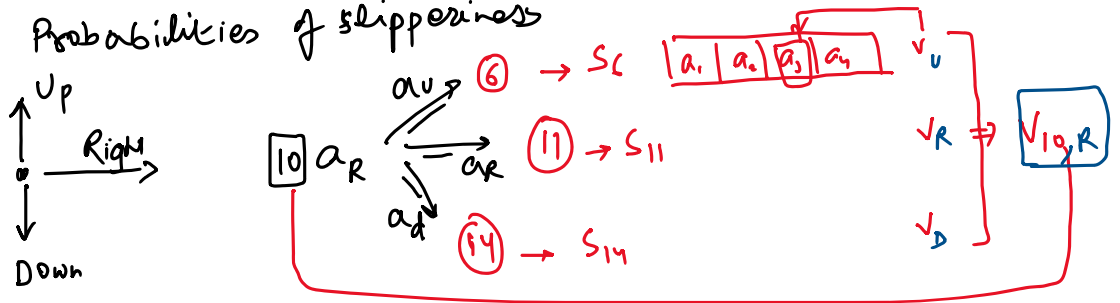
Actor - Critic

TRPO, PPO, GRPO

Model based } Model Free

Model → Environment
 (Knowledge of environment)

Probabilities of slipperiness



$$\max (V_{10,L} | V_{10,D} | V_{10,R} | V_{10,U}) \rightarrow V_{\max}$$

No knowledge (No idea about Probabilities)

	a_0	a_1	a_2^R	a_3
s_0				
s_1				
$\vdots$				
s_{10}				
s_{14}				

episode	Steps
past values	Next_step
s_0	
s_{10}	
s_{14}	

> 10
 S_{15}

> 10

> 10

Bellman Eq of Temporal Difference.

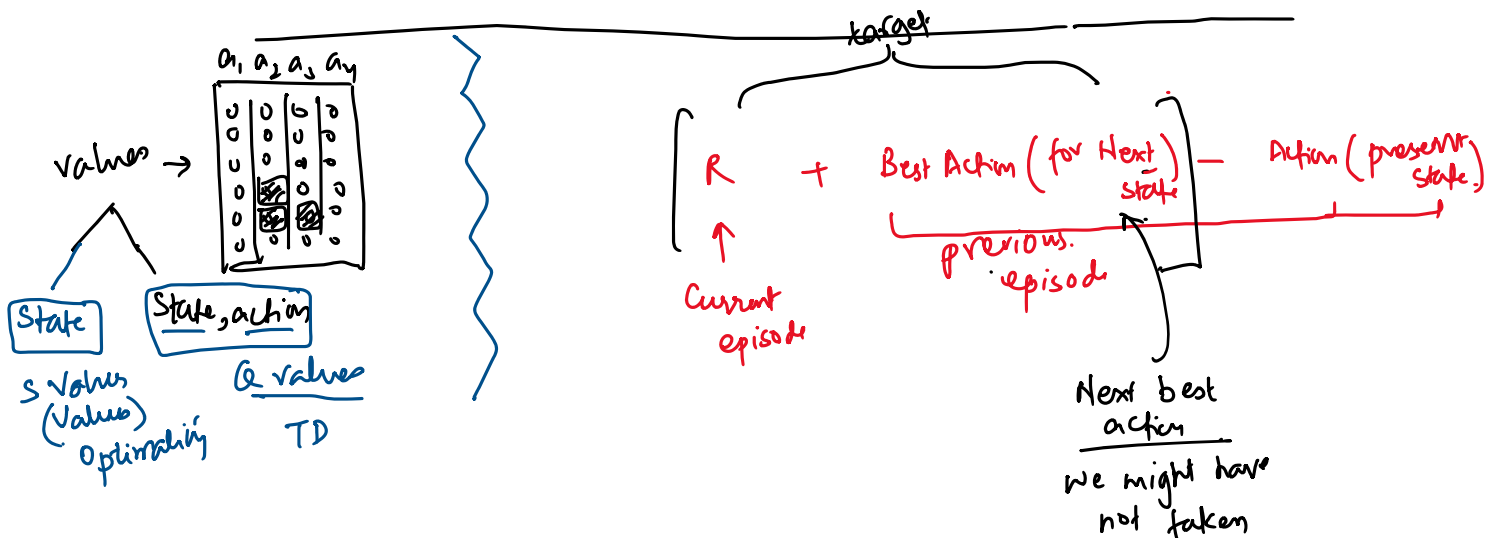
$$Q_{10,R}^{\text{New}} = Q_{10,R}^{\text{prev}} + \alpha [TD]$$

$$Q_{10,R}^{\text{New}} = Q_{10,R}^{\text{Current}} + \alpha \left[\underset{\substack{\uparrow \\ \text{Aim Best Action.}}}{\text{target}} - \underset{\substack{\uparrow \\ \text{Exploration}}}{Q_{10,R}^{\text{Current}}} \right]$$

$$Q_{10,R} = Q_{10,R} + \alpha \left[R + S_{11} \boxed{\text{max}} - Q_{10,R} \right]$$

$$Q_{10,R}^{\text{Current Episode}} = Q_{10,R}^{\text{past Ep}} + \alpha \left[R + \underset{\substack{\uparrow \\ \text{Current}}}{\gamma Q_{11,D}} - Q_{10,R}^{\text{past Episode}} \right]$$

$\uparrow$ Next Step from past episode.
 $\uparrow$ past Episode



policy $\leftrightarrow$ Behaviour (Exploration - Exploitation)
 Target (Dream) (0.1, 0.2, 0.3, ...)

ex. Cart-Pole

Big Environment

ob's space: $[-1 \dots \dots \dots +1]$
 $\uparrow$
 continuous

Cart Pole observation space = 4 ~~Discrete~~ Range $[-1, +1]$

~~Q learning~~ $\Rightarrow$ Deep Q learning

~~Q learning~~ $\Rightarrow$ Deep Q learning

Model Based

Model free.

Small Env. Big Environment

Q Learning Deep Q Learning

Deep Learning (Neural Networks)

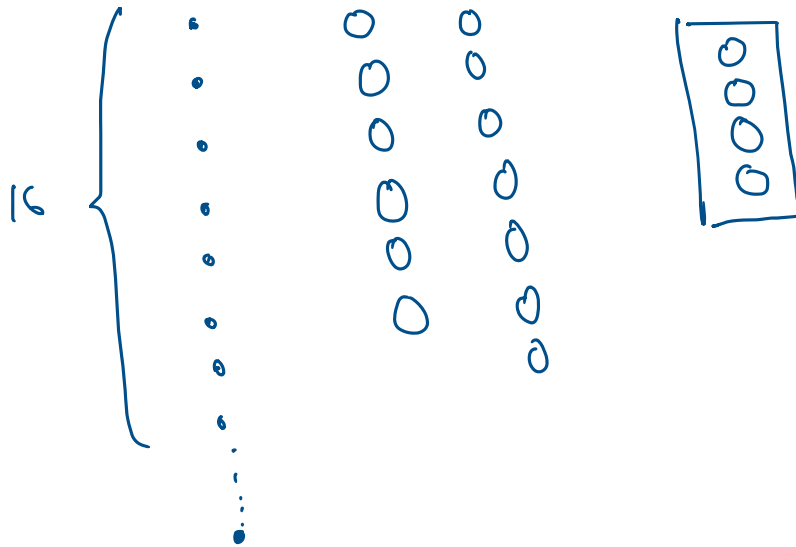
Inputs || || || Output
(Observation Space) (Action Space)

frozen lake

Cart - Pole

1

0



$$Q(S, \text{Left}) = 5$$

$$Q(S, \text{Right}) = 10$$

Behavior policy:

$S \rightarrow \text{Left}$ $\leftarrow$ actually happened

Target policy:

$S \rightarrow \text{Right}$ $\leftarrow$ policy being learned

Q-learning $\rightarrow$ "What if I take the best action next?"

SARSA $\rightarrow$ "What action did I actually take next?"

Off-policy RL becomes much clearer if you remember these two terms:

1. Behavior policy

The policy that **generates experience**.

$$\pi_b$$

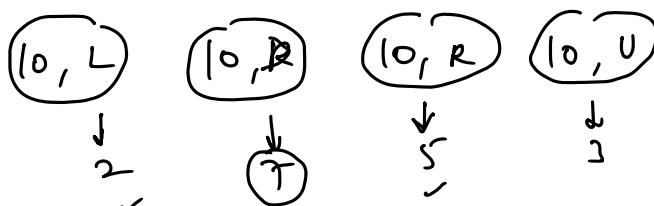
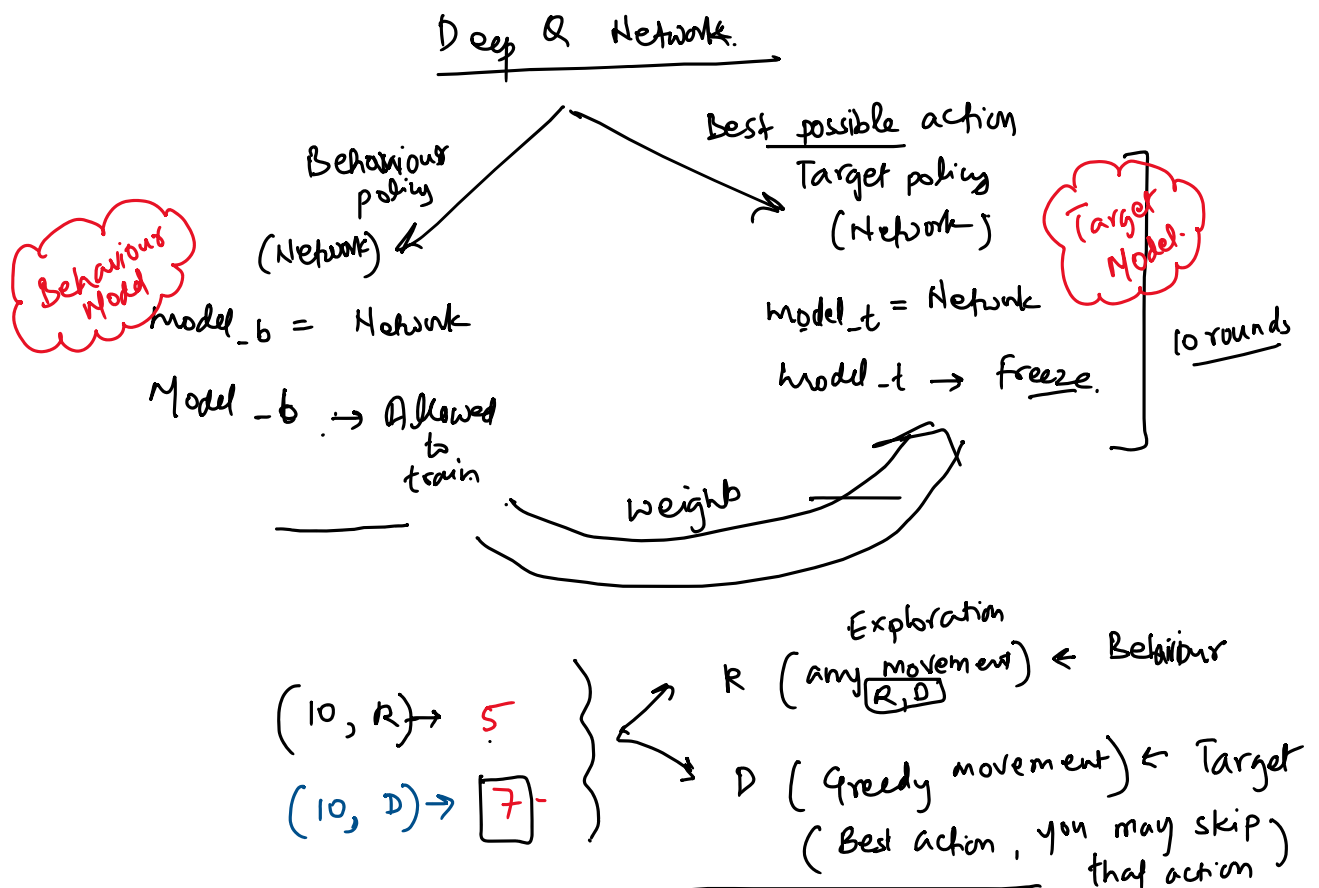
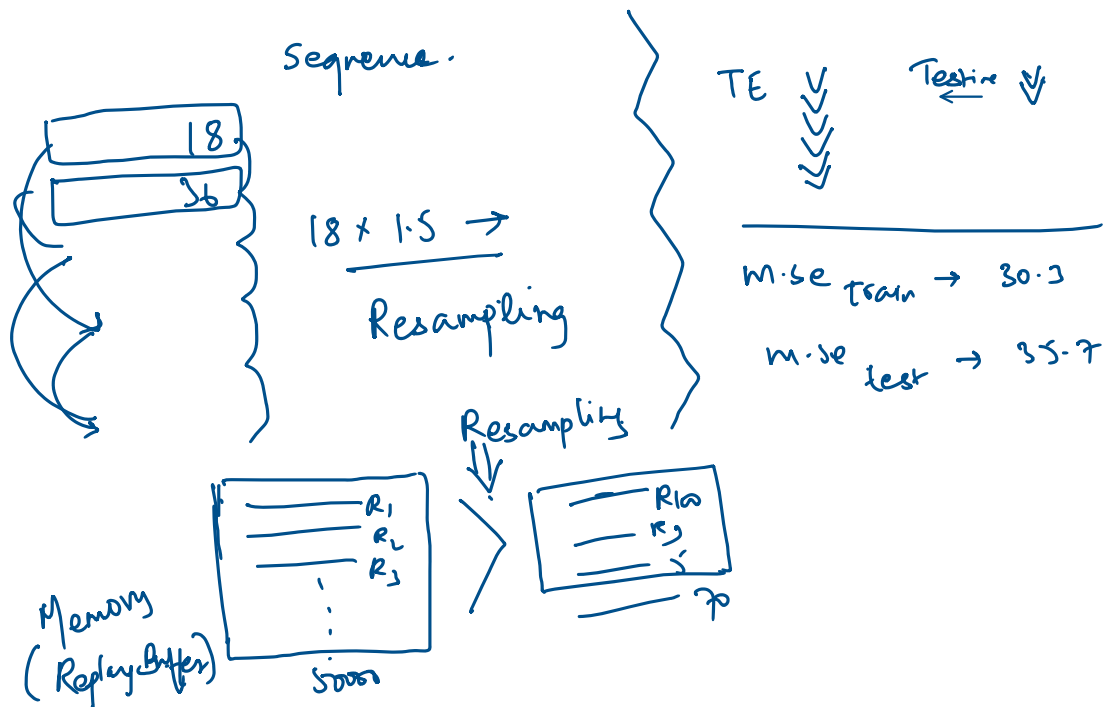
2. Target policy

The policy that we are **trying to learn**.

$$\pi_t$$

For off-policy learning:

$$\pi_b \neq \pi_t$$



Q values are going to be shaped by this

$$Q = Q + \alpha \left[\underset{\substack{\uparrow \\ \text{target}}}{R + \gamma Q_{n_x}} - \underset{\substack{\uparrow \\ \text{Behaviour}}}{Q} \right]$$

$$Q = Q + \alpha [TD_{err}]$$

12
10
...
0

```
# Choose action (epsilon-greedy)
if np.random.rand() < EPSILON:
    action = env.action_space.sample() # Random action
else:
    with torch.no_grad():
        action = torch.argmax(policy_net(torch.tensor(state)))

# Take action
next_state, reward, done, _, _ = env.step(action)
```

```
# Compute Q-values
q_values = policy_net(states).gather(1, actions)

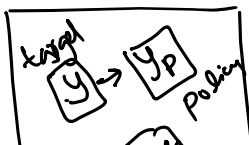
# Q(s,a) = Q(s,a) + alpha * [Reward + gamma * max(Q(s',a')) - Q(s,a)]
# Compute target Q-values using the target network
next_q_values = target_net(next_states).max(1, keepdim=True)[0] # values, indexes
target_q_values = rewards + (GAMMA * next_q_values * (1 - dones))

# Compute loss
loss = loss_fn(q_values, target_q_values.detach())
```

y - y_p
y - y_p

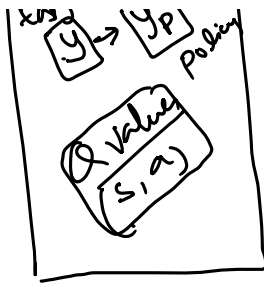
model(x) → y_p model.predict(x) → y_p

$$TD = (\text{target} - \text{q values})$$

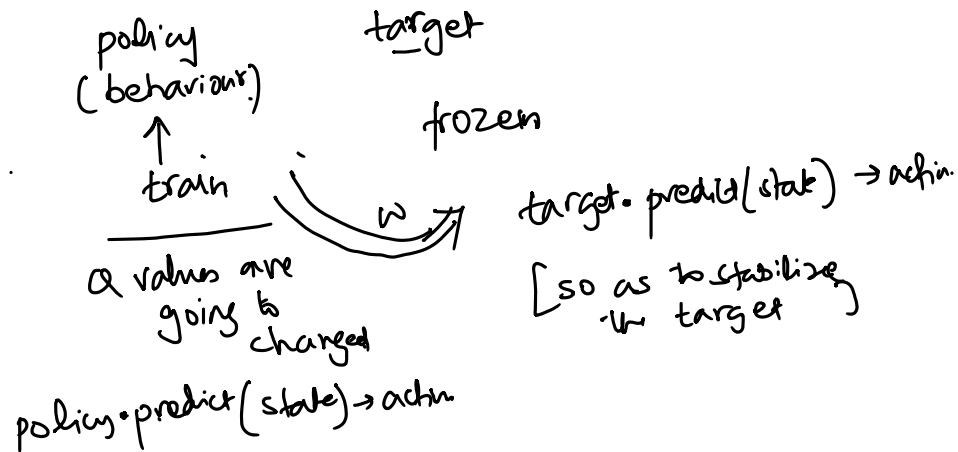


model(state) → action

model.predict(state) → action



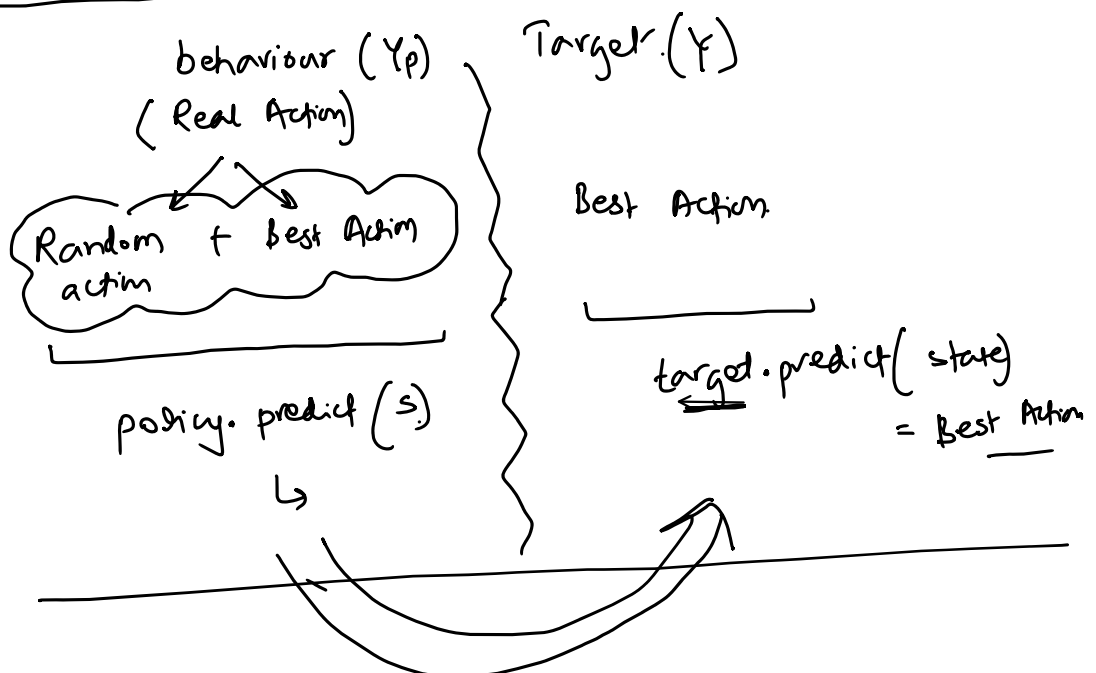
model: $\text{predict}(\text{state}) \rightarrow \text{action}$

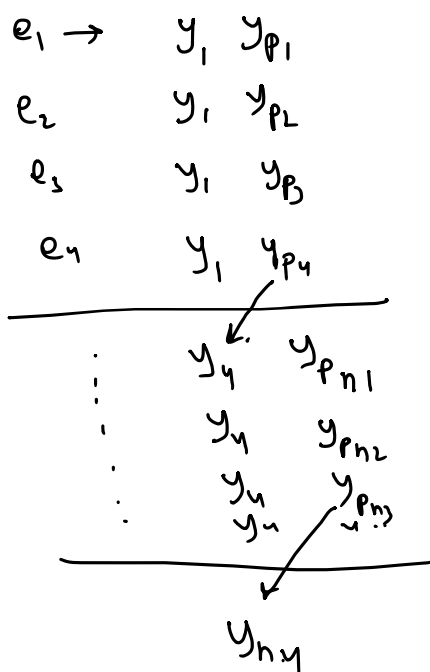
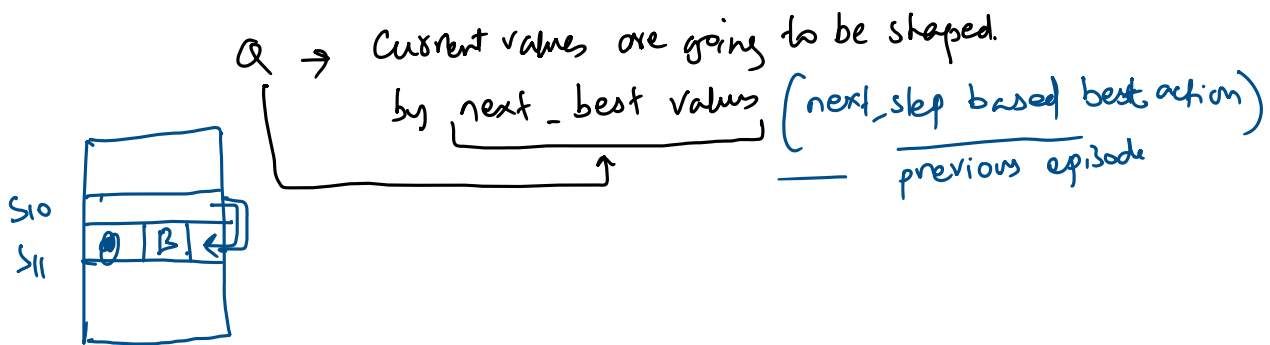


$\text{policy} \cdot \text{predict}(s) = A \rightarrow y_p \rightarrow \text{expected output}$
 $\text{target} \cdot \text{predict}(s) = A \rightarrow y \rightarrow \text{target}$

$$TD_e = y - y_p$$

$\downarrow \qquad \downarrow$
 target - Real Result
 $\downarrow \qquad \downarrow$
 $\text{target} \cdot \text{predict}(s) - \text{policy} \cdot \text{predict}(s)$





SB-1 → TensorFlow
SB-3 Library

```
memory = []

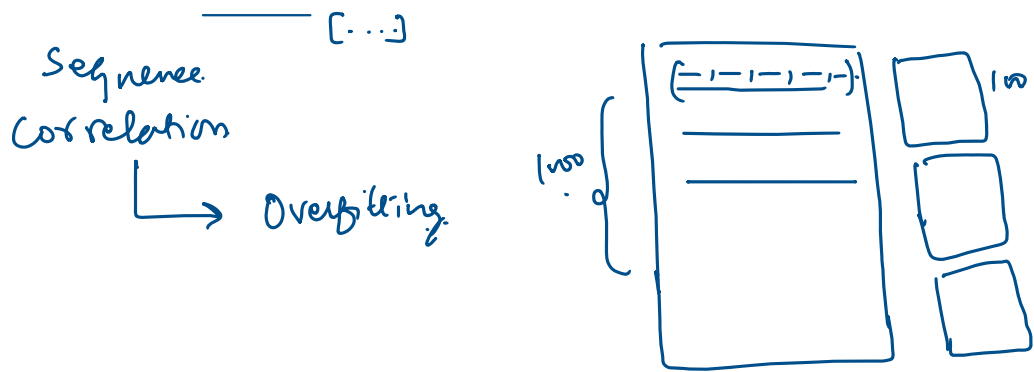
# Choose action (epsilon-greedy)
if np.random.rand() < EPSILON:
    action = env.action_space.sample() # Random action
else:
    with torch.no_grad():
        action = torch.argmax(policy_net(torch.tensor(state))).item()

# Take action
next_state, reward, done, _, _ = env.step(action)
next_state = np.array(next_state, dtype=np.float32)

# Store experience
memory.append((state, action, reward, next_state, done))
state = next_state

# Train only if enough samples are in memory
if len(memory) >= BATCH_SIZE:
    batch = random.sample(memory, BATCH_SIZE) # Sample batch
```

Handwritten notes: $t \in \{t\}$ with an arrow pointing to the `env.step(action)` line. To the right, a diagram shows a vertical list of three rows, each containing a sequence of dashes: $(-,-,-,-,-)$, $(-,-,-,-,-)$, $(-,-,-,-,-)$, followed by an ellipsis. An arrow points from the word 'loop' to this diagram.



[state, next_state, action, done, reward]

$$Q = Q + \alpha \left[\underbrace{R + \gamma Q_{\text{next}}}_{\text{target} \cdot \text{predict}()} - \underbrace{Q}_{\text{policy} \cdot \text{predict}()} \right]$$

new values

Policy Based (probabilistic)

Next Week

- REINFORCE
- Actor-Critic
- PPO, DPO
- TRPO
- GRPO

Chat GPT
Deep Seek
Drones